

Документация проекта Fraud Detection Service

1 Описание структуры проекта

```
fraud-detection-service/  
  
    src/  
        app.py  
        main.py  
        config.py  
        preprocessing.py  
        inference.py  
        explainability.py  
        gan.py  
  
    artifacts/  
        lgb_model.txt  
        lgbm_gan_aug.txt  
        gan_discriminator.pth  
        feature_order.json  
        gan_feature_order.json  
        lgbm_gan_feature_order.json  
        preprocessing_stats.json  
        model_metadata.json  
  
    data/  
        merged_test_df.parquet  
        sample_transactions.csv  
  
    tests/  
        test_api.py  
        test_preprocessing.py  
        conftest.py  
  
    Dockerfile.backend  
    Dockerfile.frontend  
    docker-compose.yml  
    requirements.txt  
    pyproject.toml  
    dvc.yaml  
    README.md
```

2 Описание файлов

2.1 Папка src

- `app.py` — Streamlit frontend;
- `main.py` — FastAPI backend;
- `config.py` — конфигурация проекта;
- `preprocessing.py` — preprocessing pipeline;
- `inference.py` — inference ML-моделей;
- `explainability.py` — SHAP explainability;
- `gan.py` — вычисление GAN anomaly score.

2.2 Папка artifacts

Содержит модели и preprocessing artifacts.

2.3 Папка data

Содержит тестовые данные и sample dataset.

2.4 Папка tests

Содержит smoke tests для API и preprocessing pipeline.

3 Описание функционала

3.1 FastAPI сервис

3.1.1 `/health`

- Метод: GET
- Назначение: проверка состояния backend-сервиса

3.1.2 `/models`

- Метод: GET
- Назначение: получение списка доступных моделей

3.1.3 `/model_info`

- Метод: GET
- Назначение: получение информации о моделях и preprocessing pipeline

3.1.4 /predict

- Метод: POST
- Назначение: inference ML-моделей
- Входные параметры:
 - model_name
 - transaction
- Возвращает:
 - fraud probability
 - prediction label
 - risk level
 - SHAP explanation

4 Preprocessing pipeline

Перед inference выполняется preprocessing входных данных:

- обработка пропусков;
- приведение типов;
- feature engineering;
- заполнение отсутствующих признаков;
- приведение признаков к train-time feature order;
- формирование итогового feature vector.

5 ML-модели

5.1 LightGBM

Основная supervised-модель для классификации мошеннических транзакций.

5.2 Hybrid LightGBM + GAN

Hybrid pipeline состоит из двух этапов:

1. GAN discriminator вычисляет anomaly score;
2. anomaly score добавляется как дополнительный признак в LightGBM.

6 SHAP Explainability

Для explainability используется SHAP.

Сервис отображает:

- top feature impacts;
- SHAP values;
- наиболее значимые признаки;
- влияние признаков на fraud prediction.

7 Streamlit приложение

7.1 Вкладка Prediction

- загрузка CSV-файла;
- preview transactions dataframe;
- выбор модели;
- выбор транзакции;
- prediction fraud probability;
- отображение risk level;
- SHAP explainability;
- отображение наиболее рискованных транзакций.

7.2 Вкладка Model info

- отображение списка моделей;
- отображение preprocessing metadata;
- отображение информации о feature space;
- отображение model versions.

8 Docker инфраструктура

Проект разворачивается через Docker Compose.

8.1 Сервисы

- fraud-api — FastAPI backend;
- fraud-ui — Streamlit frontend.

9 DVC

DVC используется для:

- версионирования моделей;
- хранения preprocessing artifacts;
- хранения feature metadata;
- воспроизводимости ML pipeline.

10 Инструкция по запуску

10.1 Требования

- Docker Desktop
- Python 3.10.4
- рекомендуется 8 GB RAM

10.2 Клонирование репозитория

```
git clone https://github.com/realtonysoprano/fraud-detection-  
service.git  
cd fraud-detection-service
```

10.3 Запуск проекта

```
docker compose up --build
```

10.4 Доступ к сервисам

Streamlit UI:

```
http://localhost:8501
```

FastAPI Swagger Docs:

```
http://localhost:8000/docs
```

11 Инструкция по использованию

1. Открыть Streamlit UI;
2. Загрузить CSV-файл (или использовать по умолчанию загруженные данные);
3. Выбрать модель;

4. Выбрать транзакцию;
5. Выполнить prediction;
6. Просмотреть fraud probability и SHAP explanation.

12 Тестирование

Для тестирования используется pytest.

12.1 Запуск тестов

```
pytest
```

Тестируются:

- preprocessing pipeline;
- FastAPI endpoints;
- корректность response structure.